Package 'golemenviro'

October 28, 2026

Title Real-time visualization of Pimoroni Enviro sensor data from Prometheus

Version 0.0.0.9000

Description A Shiny application, built as a golem package, that displays the readings of Pimoroni Enviro and Enviro+ sensors exported to Prometheus: gauges with the latest readings, and a time series chart per sensor.

License GPL-3

URL https://sedzinfo.github.io/golemenviro/, https://github.com/sedzinfo/golemenviro

BugReports https://github.com/sedzinfo/golemenviro/issues

Encoding UTF-8

Roxygen list(markdown = TRUE)

Imports config,
golem,
htmltools,
httr,
jsonlite,
plotly,
shiny,
shinydashboard,
stats,
zoo

Suggests testthat (>= 3.0.0)

Config/testthat/edition 3

Author Dimitrios Zacharatos [aut, cre]

Maintainer Dimitrios Zacharatos <dzacharatos@yahoo.com>

Config/roxygen2/version 8.1.1

Contents

get_data 2
pimoroni 3
plot_gauge 4
plot_time_series 4

Index **6**

`get_data` *Get Data from Prometheus*

Description

Downloads the readings of all the sensors of a Pimoroni Enviro or Enviro+ from Prometheus, at about 1000 points in time between `start_time` and `end_time`.

Usage

```
get_data(
  start_time = format(Sys.time() - 60 * 60 * 24 * 7, "%Y-%m-%dT%H:%M:%SZ", tz =
    "UTC"),
  end_time = format(Sys.time(), "%Y-%m-%dT%H:%M:%SZ", tz = "UTC"),
  urlq
)
```

Arguments

`start_time`	start time, in UTC, in this format: "2025-01-31T11:48:08Z"
`end_time`	end time, in UTC, in this format: "2025-02-07T11:48:33Z"
`urlq`	Prometheus url

Value

A data frame with the Date and the Timestamp of the readings and one column per sensor, or `NULL`, with a message, when the server cannot be reached or sends no data. Sensors whose readings never change are left out, as the device does not have them.

Examples

```
prometheus_url <- c(
  "http://pip1.crabdance.com:1507/api/v1/query_range",
  "http://pip1.crabdance.com:1505/api/v1/query_range"
)
start_time <- format(Sys.time() - 60 * 60 * 24 * 7, "%Y-%m-%dT%H:%M:%SZ", tz = "UTC")
end_time <- format(Sys.time(), "%Y-%m-%dT%H:%M:%SZ", tz = "UTC")
filtered_df <- get_data(start_time = start_time, end_time = end_time, urlq = prometheus_url[2])
head(filtered_df)
filtered_df <- get_data(start_time = start_time, end_time = end_time, urlq = prometheus_url[1])
head(filtered_df)
```

`pimoroni` *Run the Pimoroni Enviro app*

Description

Starts a Shiny app with the readings of Pimoroni Enviro and Enviro+ sensors exported to Prometheus: gauges with the latest readings in the Current tab, and a chart per sensor in the Time Series tab. The app has a dark mode, switched by the button at the right of the navbar.

Usage

```
pimoroni(
  prometheus_url,
  timezone = "Europe/Bucharest",
  analytics = FALSE,
  onStart = NULL,
  options = list(),
  enableBookmarking = NULL,
  uiPattern = "/",
  ...
)
```

Arguments

`prometheus_url`	Character vector of Prometheus query_range urls, one per device, such as "http://pip1.crabdance.com:
`timezone`	The time zone the readings are shown in, while the app runs.
`analytics`	`TRUE` to add Google Analytics, through the cookie banner shared with the sites of sedzinfo.github.io, which loads it only after the visitor accepts. Meant for the public deployment (see `app.R`); off by default, so that running the app yourself sends nothing.
`onStart`	See `shiny::shinyApp()`.
`options`	See `shiny::shinyApp()`.
`enableBookmarking`	See `shiny::shinyApp()`.
`uiPattern`	See `shiny::shinyApp()`.
`...`	Further options passed to `golem::with_golem_options()`.

Value

A `shiny::shinyApp()` object. Printing it, or calling `pimoroni()` at the console, opens the app in the browser.

Examples

```
if (interactive()) {
  pimoroni(prometheus_url = c(
    "http://pip1.crabdance.com:1507/api/v1/query_range",
    "http://pip1.crabdance.com:1505/api/v1/query_range"
  ))
}
```

plot_gauge *Plot Gauge*

Description

Plots the last reading of one sensor as a gauge, with the ideal range in green and a short note on what the gas and particle sensors measure. The gas sensors have no fixed range, so their gauge is scaled to all the readings of `data`: it spans the lowest to the highest reading, widened by one standard deviation, the green band is the mean plus or minus one standard deviation, and the delta is the difference from the mean.

Usage

```
plot_gauge(data, variable = "Temperature")
```

Arguments

`data`	data frame from get_data(); the gauge shows its last reading
`variable`	one of "Temperature" "Humidity" "Pressure" "Lux" "Proximity" "NH3" "Reducing" "Oxidising" "PM1" "PM2.5" "PM10"

Value

A plotly object.

Examples

```
prometheus_url <- c(
  "http://pip1.crabdance.com:1507/api/v1/query_range",
  "http://pip1.crabdance.com:1505/api/v1/query_range"
)
filtered_df <- get_data(urlq = prometheus_url[1])
plot_gauge(data = filtered_df, variable = "Temperature")
plot_gauge(data = filtered_df, variable = "Humidity")
plot_gauge(data = filtered_df, variable = "PM2.5")
```

plot_time_series *Plot Time Series*

Description

Plots the readings of one sensor, with their moving average, their mean, and a summary of the last reading, mean, minimum, maximum, standard deviation, range and dates on the right.

Usage

```
plot_time_series(data, variable = "Temperature", scale = "", k = 30)
```

Arguments

data	data frame from get_data()
variable	one of "Temperature" "Humidity" "Pressure" "Lux" "Proximity" "NH3" "Reducing" "Oxidising" "PM1" "PM2.5" "PM10"
scale	title of the y axis
k	integer width of the rolling window. Must be odd for rollmedian

Value

A plotly object.

Examples

```
prometheus_url <- c(
  "http://pip1.crabdance.com:1507/api/v1/query_range",
  "http://pip1.crabdance.com:1505/api/v1/query_range"
)
filtered_df <- get_data(urlq = prometheus_url[1])
plot_time_series(data = filtered_df, variable = "Temperature")
plot_time_series(data = filtered_df, variable = "Humidity")
plot_time_series(data = filtered_df, variable = "Pressure")
plot_time_series(data = filtered_df, variable = "PM2.5")
```

Index

∗ **gauge**
    plot_gauge, 4
∗ **plot**
    plot_gauge, 4
    plot_time_series, 4
∗ **prometheus**
    get_data, 2
∗ **query**
    get_data, 2
∗ **timeseries**
    plot_time_series, 4

get_data, 2
golem::with_golem_options(), *3*

pimoroni, 3
plot_gauge, 4
plot_time_series, 4

shiny::shinyApp(), *3*